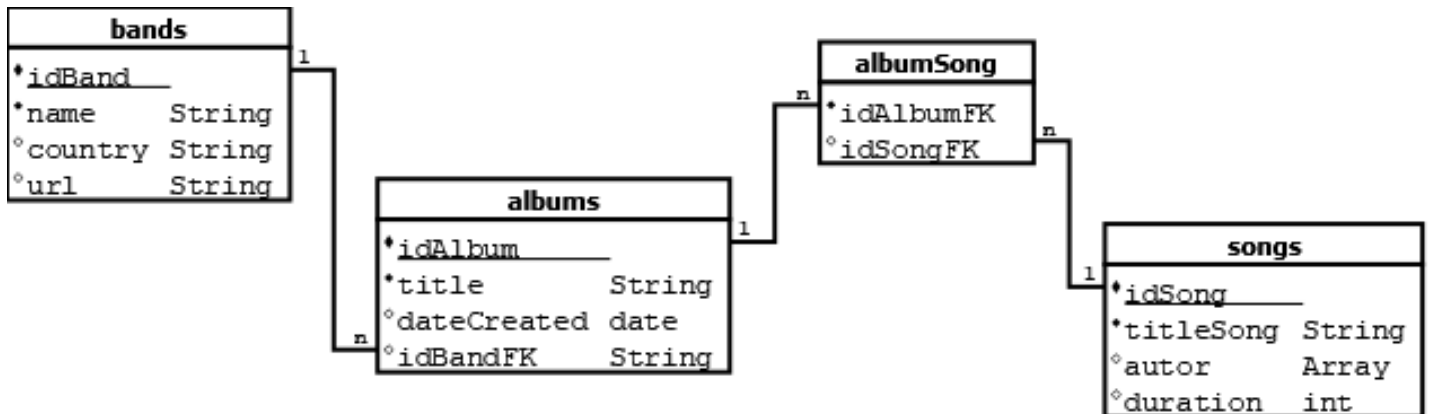
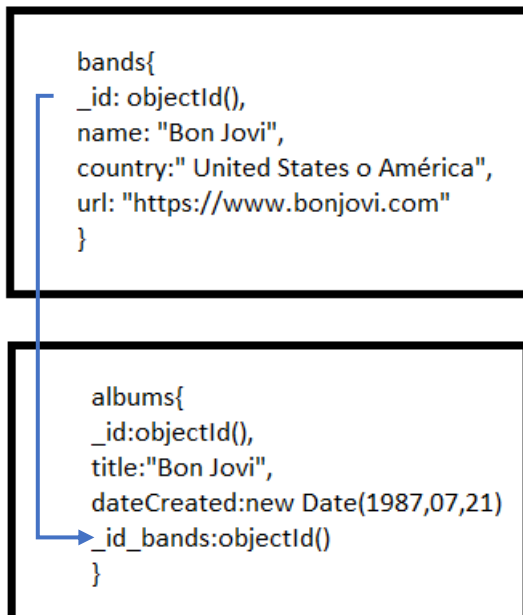


SUBDOCUMENTOS



Para un esquema donde se necesita mantener información relacionada, lo que comúnmente llamamos relaciones entre tablas en un modelo relacional, en una base de datos de mongoDB tiene una representación similar que se puede manejar de formas diferentes:

Colecciones separadas relacionadas



Documentos embebidos en una colección

Si el documento embebido es solo uno, podemos tener una similitud con una base de datos relacional y una **relación uno a uno**, que se representaría así:

```
bands{
  name: 'Bon Jovi',
  country: 'Estados Unidos de América',
  url: 'https://www.bonjovi.com/',
  albums:
  {
    title: 'Bon Jovi',
    dateCreated: new Date(1984,01,21)
  }
}
```

Si hay varios documentos embebidos, podemos tener una similitud con una **relación uno a muchos**, que se representaría así:

```
bands{
  name: 'Bon Jovi',
  country: 'Estados Unidos de América',
  url: 'https://www.bonjovi.com/',
  albums: [
    {
      title: 'Bon Jovi',
      dateCreated: new Date(1984,01,21)
    },
    {
      title: 'Otro album',
      dateCreated: new Date(1987,01,21)
    }
  ]
}
```

|

Podemos tener tantos documentos embebidos como sean necesarios, siempre y cuando no se afecte el tamaño máximo de las colecciones mongoDB que es 16 MB

```
bands:{
  name: 'Bon Jovi',
  country: 'Estados Unidos de América',
  url: 'https://www.bonjovi.com/',
  album: [
    {
      title: 'Bon Jovi',
      dateCreated: new Date(1984,01,21),
      songs: [
        {
          title: 'Runaway'
        },
        {
          title: 'Roulett'
        }
      ]
    }
  ]
}
```

Operaciones CRUD sobre una colección con documentos embebidos

Mostrar datos del documento embebido

Ej: nombre y país de origen de la banda y títulos de los albums de la banda Bon Jovi (Los campos de los documentos embebidos deben escribirse entre comillas " "):

```
db.bands.find({name:"Bon Jovi"},{_id:0,name:1,country:1,"albums.title":1})
```

Modificar el valor de un campo en un documento embebido

Ej: Modificar por un id específico el nombre de la banda a Guns & Roses y el álbum de la banda a Welcome to the Jungle.

```
db.bands.updateOne({_id: ObjectId("635f1cd250e7fa0477df8228")},{ $set:{name:"Guns & Roses"},"albums.title":"Welcome to the Jungle"}})
```

Borrar documentos a partir del campo de un subdocumento

Ej: Borrar el primer registro cuyo título de álbum sea Bon Jovi

```
db.bands.deleteOne({"albums.title":"Bon Jovi"})
```

|

Agregar a un subdocumento existente un nuevo documento

Ej: Agregar a la banda Bon Jovi un nuevo álbum con título 7800° Fahrenheit y fecha creación 12 abril 1985.

```
db.bands.updateOne({name: "Bon Jovi" }, { $set: { "albums.2.title":"7800° Fahrenheit", "albums.2.dateCreated":new Date(1985,04,12)}})
```

Ej: Agregar al tercer álbum de Bon Jovi 2 canciones

```
db.bands.updateOne({ name: "Bon Jovi" }, { $set: { "albums.2.songs":[{"title:"Canción 1"}, {"title:"Cancion 2"}]}})
```

Borrar subdocumentos

Ej: Borrar canciones asociadas al álbum 7800° Fahrenheit

```
db.bands.updateOne({"albums.title": "7800° Fahrenheit" }, { $unset: { "albums.2.songs":[{"title:"Apetite for destruction"}]}})
```

Actividad

1. Agregar a la base de datos 5 bandas (datos libres)
2. Agregar a cada banda mínimo 2 albums musicales de su discografía (subdocumento)
3. Agregar a cada álbum mínimo 5 canciones (subdocumento songs)
4. Mostrar cada banda completa con su discografía relacionada
5. Borrar un campo del subdocumento albums (a elección)
6. Borrar todo el subdocumento songs de uno de los albums de una banda (a elección)